

OFFENSIVE SECURITY LAB INVESTIGATION REPORT

Lab 1 — Lab Assurance, HTTP Mapping and Attack-Surface Reconnaissance

Course: Offensive Security I

Assessment/Lab: Lab 1

Investigation Type: Web Application Security / Attack-Surface Reconnaissance

Environment: Isolated Local Training Laboratory

Date: 26 August 2026

Prepared by: Muhammad Modu Kunjo

1. Executive Summary

This report documents the investigation and validation of an isolated web-security training environment used for **Lab 1 — Lab Assurance, HTTP Mapping and Attack-Surface Reconnaissance**.

The investigation focused on establishing a controlled laboratory environment, deploying the Mutillidae vulnerable web application, validating the DVWA installation, inspecting HTTP requests and responses, identifying technologies and application characteristics, and documenting the attack surface.

During the setup and investigation, several configuration and connectivity issues were encountered. These included Docker daemon connectivity problems, an unavailable docker.service, an initially incorrect DVWA path, and a database connection failure within DVWA. Each issue was investigated and corrective actions were taken.

The investigation ultimately established a functioning training environment. The web application stack was identified as **Apache 2.4.68 (Debian)** with **PHP 8.5.9**. DVWA was confirmed to be reachable, returning an HTTP 302 Found response, and the database was subsequently reset successfully.

An information-disclosure condition was also observed in application source code where an HTML comment contained database-password-related information. Although this was identified within an intentionally vulnerable training application, the observation demonstrates the security risk of leaving credentials, secrets, or sensitive implementation information in client-delivered source code.

All testing described in this report was performed against the controlled laboratory environment. No production system was tested.

2. Objectives

The objectives of this investigation were to:

1. Validate that the web-security laboratory was appropriately isolated.
2. Establish the Mutillidae and DVWA vulnerable applications.
3. Verify application availability and normal behaviour.
4. Map HTTP requests and responses.
5. Identify technologies exposed by the applications.
6. Identify relevant application inputs, endpoints, cookies and headers.
7. Establish a baseline attack surface before vulnerability testing.
8. Capture and document relevant evidence.
9. Record errors, troubleshooting steps and corrections.
10. Produce a reproducible investigation record.

3. Scope

The investigation was restricted to the intentionally vulnerable applications deployed within the local security-training environment.

In Scope

- Mutillidae
- DVWA
- Local Docker environment
- Local VM networking
- HTTP requests and responses
- Application source available to the browser
- Application technology fingerprinting
- ZAP interception activity
- Configuration and deployment troubleshooting

Out of Scope

- Production systems
- Public websites
- Third-party infrastructure
- Unauthorised external systems
- Real user information
- Destructive testing outside the laboratory

4. Rules of Engagement

Testing was conducted under the following principles:

- Testing was limited to the authorised training environment.
- The vulnerable applications were intentionally deployed for security education.
- Network isolation was maintained during testing.
- Normal application behaviour was established before attempting security testing.
- Evidence was documented throughout the investigation.
- Troubleshooting activities were recorded rather than omitted.
- No attempt was made to affect systems outside the laboratory.

5. Laboratory Environment

The laboratory consisted of a Linux-based security-testing environment with vulnerable web applications deployed using Docker.

5.1 Network Configuration

The VM was configured using a Host-Only network.

Observed interface:

eth0

192.168.56.101/24

Docker networking was also observed:

172.18.0.1/16

172.19.0.1/16

The following local services were observed:

MariaDB: 127.0.0.1:3306

Docker Proxy: 127.0.0.1:82

The use of a Host-Only network helped maintain separation between the training environment and external networks.

6. Mutillidae Deployment

The Mutillidae Docker environment was obtained and deployed using Git and Docker Compose.

The deployment commands used were:

```
git clone https://github.com/webpwnized/mutillidae-docker.git
```

Followed by:

```
docker compose -f .build/docker-compose.yml up --build --detach
```

The deployment created the following application components:

ldap

database

www

These components provided the application and supporting services required for the laboratory.

7. Deployment Troubleshooting

The initial deployment process encountered an error indicating that the Docker daemon could not be reached.

Error

Cannot connect to Docker daemon

An attempt was made to start the Docker service:

```
sudo systemctl start docker
```

This produced:

Failed to start docker.service:

Unit docker.service not found

The Docker environment was subsequently inspected, and the installed Docker version was observed as:

Docker 28.5.2+dfsg4

The Docker Compose environment was eventually built successfully.

Security/Learning Significance

This troubleshooting stage demonstrated an important principle of security testing:

A failed tool or service does not necessarily indicate that the target is unavailable.

The tester must distinguish between:

- Target failure
- Tool failure
- Configuration failure
- Network failure
- Authentication failure
- Environmental failure

Documenting these distinctions improves the reliability and reproducibility of an investigation.

8. DVWA Discovery

During the initial application testing, DVWA was not immediately accessible at the expected location.

An initial request produced a:

Not Found

condition.

The web root was subsequently inspected and the following directory was identified:

/var/www/html/DVWA/

This established that DVWA was present within the web server's document root.

9. DVWA HTTP Validation

The application was tested using curl.

The following command was used:

```
curl -I http://127.0.0.1:8080/DVWA/
```

The response returned:

HTTP/1.1 302 Found

This demonstrated that the DVWA application was reachable and responding to HTTP requests.

Evidence: DVWA returned HTTP 302 Found.

Purpose: Confirm application discovery and HTTP reachability.

Significance: The response established that the application existed at the discovered location and was responding to requests.

10. Technology Fingerprinting

The investigation identified the following server-side technologies:

Technology	Observed Version
Web Server	Apache 2.4.68
Operating System Distribution	Debian
Server-side Language	PHP

Technology	Observed Version
-------------------	-------------------------

PHP Version	8.5.9
-------------	-------

The application also exposed browser information similar to:

Mozilla/5.0 (X11; Linux x86_64)

AppleWebKit/537.36

Chrome/146.0.0.0

Safari/537.36

Security Significance

Technology fingerprinting assists a penetration tester in understanding the target environment.

However, version information should be treated carefully. Excessive technology disclosure can help an attacker identify:

- Potentially vulnerable components
- Applicable exploits
- Configuration weaknesses
- Technology-specific attack techniques

Therefore, unnecessary information disclosure should be minimised in production systems.

13. HTTP Headers

Several HTTP headers were observed during the investigation.

Examples included:

Cache-Control: public

X-XSS-Protection: 0

Strict-Transport-Security: max-age=...

These headers formed part of the application's HTTP security baseline.

The presence or absence of a particular security header should not be interpreted in isolation. Header configuration should be assessed together with:

- Application behaviour
- Browser behaviour
- Authentication controls
- Session management
- Content handling
- Server configuration